

Cattle Breed Classifier Web App

One-page repo summary based on README, Flask app, frontend assets, docs, config, and training scripts.

What It Is

A web app that predicts cattle breed from an uploaded image or image URL using a PyTorch image classification model. The repo also includes training and evaluation code for the ResNet-50-based classifier.

Who It's For

Primary persona: someone who wants a quick cattle breed prediction from a photo in the browser, such as a student, hobbyist, or practitioner exploring Indian cattle breed identification.

What It Does

- Accepts either a local image upload or a remote image URL for classification.
- Returns top predictions with class names, raw outputs, and probabilities via `/api/classify`.
- Shows sample image URLs from `config.yaml` for quick testing in the UI.
- Serves a React single-page interface with Home and About routes.
- Loads app metadata from `/config`, including title, description, about text, and repo link.
- Exposes `/api/classes` to return the available label set from `models/classes.txt`.
- Includes standalone training and evaluation scripts for the underlying vision model.

How It Works

Frontend: `static/index.html` loads CDN-hosted React/Reactstrap and `static/js/main.jsx`. The UI fetches `/config`, lets users choose a sample URL or upload a file, then calls the backend.

Backend: `app.py` starts Flask, downloads `models/cattle_breed_classifier_full_model.pth` and `models/classes.txt` from Google Drive when missing, then serves `/api/classify`, `/api/classes`, `/config`, `/ping`, and static assets.

Inference: uploaded bytes or a fetched URL image are opened with Pillow, normalized with torchvision transforms, scored by the loaded PyTorch model, and returned as the top 3 predictions.

Training pipeline: `src/dataset_loader.py`, `src/model.py`, `src/train.py`, and `src/evaluate.py` define ImageFolder loading, a ResNet-50 classifier head, training, and accuracy evaluation.

How To Run

- Install Python dependencies: `pip install -r requirements.txt`.
- Start the app from the repo root: `python app.py`.
- Open the local Flask URL shown in the terminal (defaults to port `5001` unless `PORT` is set).
- On first run, expect the app to download the model weights and class list into `models/` if they are missing.

Repo Gaps / Notes

Exact supported breeds count is inconsistent in the repo: docs mention 26 classes, while evaluation code uses 30. Local development prerequisites beyond `requirements.txt` are not documented in detail.

Not found in repo: authentication, persistence/database, test suite, container config, or a dedicated frontend build step.